CSE 3100 Systems Programming Spring 2026

Exam #2

This is a 110-minute exam including MCQs and coding. *Please read every question carefully.*
Please read the following list of important items on exam policy and make sure you understand it.

- During the exam, you cannot communicate with and/or obtain help from people other than TAs and instructors on exam questions. Particularly, you cannot use any messaging applications.
- During the exam, you can only access data/files on HuskyCT, your github codespaces, man pages and files on your own laptop and on the lab computer you use. You cannot access data/files on other computers/servers. You cannot use your phone. You cannot use Chat-GPT or other generative AI.
- Return the question paper before you leave the exam room.
- After the exam, you cannot discuss/disclose exam problems and/or share your solutions with students who have not taken the exam.
- Failure to follow the rules will result in a zero on the exam and may cause you to fail the course.
- Unless mentioned, assume all code snippets contain the necessary header files for MCQs and free response questions.
- Submitting to Gradescope after leaving the exam room is not allowed.

Full Name: _____ NetID: _____

Lab Section: _____ Date: _____

Signature: _____

Multiple Choice

1. In a scenario where a parent process has terminated but its child process is still running, what happens to the child process? (3)

   A. The child process is terminated automatically

   B. The child process continues to run as an orphan process.

   C. The child process is paused until another parent process adopts it.

   D. The child process is terminated and returns an as error code to the operating system.

2. What is the logical error with the pseudo code provided (3)

```
1. Fork a child process.
2. If fork is successful, check if it's the child process.
3. If it's the child process:
   a. Execute a command using execvp, e.g., "ls -l".
   b. Print a message after the execvp call.
4. If it's the parent process:
   a. Wait for the child process to finish.
   b. Print a message indicating the child process has finished.
5. End of program.
```

   A. The code fails to wait for the parent process to finish.

   B. The child process attempts to execute code after execvp call.

   C. There is nothing wrong with the pseudo code provided here.

   D. Both A and B.

3. Read the following code: (3)

```c
int main() {
    int a = fork();
    int b = fork();
    return 0;
}
```

   In the code above, how can the child process of the child process of the main parent process be identified?

   A. $a == 0,\ b > 0$

   B. $a > 0,\ b > 0$

C. $a > 0,\ b == 0$

D. $a == 0,\ b == 0$

4. What is the output for the code below ? (3)

```
int main() {
        execl("ls","ls","-l",NULL);
        printf("I live with the confidence that
                if I believe in myself, the money will follow.");
  return 0;
}
```

A. It will print all the files in the folder without any information about permissions.

B. It will print all the files in the folder with information about permissions.

C. It will print all the files in the folder with information about permissions and will also execute the print statement.

D. None of the above

5. Which of the following statements best describes the differences between `fopen()` and `open()` in C? (3)

A. `fopen()` and `open()` are both part of the C standard library, but `fopen()` provides extra options for binary mode that `open()` does not support.

B. `fopen()` is declared in `<stdio.h>` and returns a pointer to a `FILE` structure that supports buffered I/O, whereas `open()` is declared in `<fcntl.h>` and returns an integer file descriptor used for unbuffered I/O.

C. `fopen()` can be used to open both files and directories, while `open()` can open only files.

D. Both `fopen()` and `open()` are system calls that directly interact with the kernel and provide identical functionality.

6. Consider the text file test.txt with the contents below (3)

```
pizza
```

The above file is only one line. Consider the code below, that reads the contents of the file above.

```
1 int main() {
2   int fd = open("test.txt", O_RDWR);
3   char buff[10];
4   int v = read(fd, buff, 10);
5   return 0;
6 }
```

Choose all the correct options.

A. The value of v is 10 as we are reading 10 bytes with read().

B. The value of v is 6

C. The read will not work because we only have O_RDWR instead of O_RDONLY.

D. The value of v will be zero because of the not using O_RDONLY.

7. Consider the code below (3)

```
1 int main() {
2         printf("Jambalaya!!!\n");
3   return 257;
4 }
```

This program is run using execl in another program and the exit status was captured. The code ran successfully. Choose all the correct options.

A. WIFEXITED=1 and WIFEXITSTATUS=256, because exit status is only between 0 and 256.

B. WIFEXITED=1 and WIFEXITSTATUS=257, becuase the program has returned 257.

C. WIFEXITED=0 and WIFEXITSTATUS=1, becuase the program has returned 257 and exited correctly.

D. WIFEXITED=1 and WIFEXITSTATUS=1, becuase the program has returned 257 and exited correctly.

8. Select the correct statements below regarding pipes. (3)
int p[2]; p will hold the file descriptors of pipe.

   A. p[0] is used to read

   B. p[1] is used to read

   C. p[1] is used to write

   D. p[0] is used to write

9. Select all the correct statements. (3)

   A. Pipe is a unidirectional data channel

   B. C compiler will throw a syntax error if we use one pipe for bi-directional communication between two proceses.

   C. We can technically use only one pipe to communicate between more than two processes, if it is unidirectional.

   D. We can technically use only one pipe to communicate between more than two processes, if it is bi-idirectional.

10. Select all the correct statements. (3)

   A. Pipes can only be used between processes that are related.

   B. If you redirect a file descriptor using dup2 in parent, it will also be redirected in child process, if the child process is created after redirecting

   C. If you redirect a file descriptor using dup2 in parent, it will also be redirected in child process,no matter where the child process is created.

   D. dup() alone is enough to redirect a file descriptor.

Free response questions

11. What is the difference between wait() and waitpid()? What happens to the calling and callee process during and after wait()? Which function among the above can be used to wait on the parent ? (5)

12. Consider zip command, which is used to compress files into a zipped folder. It is located in the directory /usr/bin and is also on the path. What are all the ways to run the below command using exec ? (5)

```
zip files.zip test.txt apple.c ball
```

13. What file descriptors are open at the exit of the program? (5)

```c
int main() {
  int fd = open("test.txt", O_RDWR | O_CREAT);
  int dfd = dup(fd);
  dup2(fd, 0);
  dup2(dfd, 1);
  char *s1 = "Hello";
  write(fd, s1, strlen(s1));
  close(fd);
  return 0;
}
```

14. Use read() system call to read till the comma (,) exclude comma, in the below contents of file test.txt and store it in a buffer. (5)

```
In Another Life, I Would Have Really Liked
Just Doing Laundry and Taxes With You.
```

Coding

(50 points) Soup du jour

In this exercise, you will write a C program **from scratch** (no starter code provided).

Your program will involve **three processes**: a *Parent*, *Child 1*, and *Child 2*, communicating via pipes. The program takes a single character as a command-line argument and passes it through the processes as follows:

Step 1. Parent → Child 1: Parent reads the character from command line arguments and sends it to Child 1.
Example: Parent sends `'a'` to Child 1.

Step 2. Child 1 → Child 2: Child 1 receives the character and sends the *next* character to Child 2.
Example: Child 1 receives `'a'`, sends `'b'`.

Step 3. Child 2 → Parent: Child 2 receives the character and sends the *next* character to Parent.
Example: Child 2 receives `'b'`, sends `'c'`.

Step 4. Parent → Child 2: Parent receives the character and sends the *next* character back to Child 2.
Example: Parent receives `'c'`, sends `'d'`.

Step 5. Child 2 → Child 1: Child 2 receives the character and sends the *next* character to Child 1.
Example: Child 2 receives `'d'`, sends `'e'`.

Step 6. Child 1 → Parent: Child 1 receives the character and sends the *next* character to Parent.
Example: Child 1 receives `'e'`, sends `'f'`.

Step 7. Parent: Parent receives the character and we are done.
Example: Parent receives `'f'`.

Game over.

Your code should print what the process is reading and writing at each exchange. Make sure to name your code solution.c Below is an example output:

```
$ ./solution a
Child 2 read:b
Child 2 wrote:c
Child 2 read:d
Child 2 wrote:e
Child 1 read:a
Child 1 wrote:b
Child 1 read:e
Child 1 wrote:f
Parent wrote:a
Parent read:c
Parent wrote:d
Parent read:f
```

Note that the print statements can be printed in different order by the OS. But the overall result should be as given in the code.
Parent should start with character `'a'` and end with `'f'` for the above test case

Another test case:

```
$ ./solution !
Child 2 read:"
Child 2 wrote:#
Child 2 read:$
Child 2 wrote:%
Child 1 read:!
Child 1 wrote:"
Child 1 read:%
Child 1 wrote:&
Parent wrote:!
Parent read:#
Parent wrote:$
Parent read:&
```

Parent should start with character `'!'` and end with `'&'` for the above test case.
Below is the command for valgrind:

```
$. valgrind --track-fds=yes ./solution
```

Look for "File Descriptors", there will be three of them , one for each process.
Submit solution.c to gradescope.